# Entrega — Semana 3

**Actividad:** usar Docker para construir dos contenedores que estén comunicados entre sí.

**Autor:** Sebastián Velásquez **Repositorio:** https://github.com/SebasVelasquez/docker-semana3 **Fecha de ejecución de las pruebas:** 10 de noviembre de 2026

---

## 1. Qué se construyó

Dos contenedores que trabajan juntos:

| # | Contenedor | Qué es | Imagen |
|---|---|---|---|
| 1 | `semana3-api` | Una API hecha en Python con Flask, que guarda y consulta mensajes | Construida a medida con `api/Dockerfile` |
| 2 | `semana3-db` | Una base de datos PostgreSQL, que almacena esos mensajes | `postgres:16-alpine` (imagen oficial) |

La API no guarda nada en su propia memoria: **todo lo escribe y lo lee del otro contenedor**. Si la base de datos no estuviera accesible, la API no podría responder. Eso es lo que demuestra que los dos se comunican.

---

## 2. Cómo se comunican (la idea central)




Tres cosas que hacen posible la comunicación:

**a) Están en la misma red.** En `docker-compose.yml` se declara una red llamada `red-app` y los dos servicios se conectan a ella. Solo los contenedores de esa red se ven entre sí.

**b) Se llaman por el nombre del servicio, no por `localhost`.** Docker levanta un servidor DNS propio dentro de la red: el nombre `db` se traduce a la dirección IP del contenedor de PostgreSQL. Por eso la API se configura con `DB_HOST: db`.

> **¿Por qué no `localhost`?** Porque dentro de un contenedor, `localhost` significa *ese mismo contenedor*. Si la API buscara la base de datos en `localhost:5432`, se estaría buscando a sí misma, y ahí no hay ninguna base de datos. Es el error más común al empezar con Docker.

**c) La base de datos no se publica al exterior.** El servicio `db` no tiene sección `ports:`, así que su puerto 5432 **solo** existe dentro de `red-app`. El único puerto abierto hacia mi computador es el 5000 de la API. Esto es buena práctica de seguridad y, además, prueba que el tráfico entre ambos pasa por la red interna de Docker y no por fuera.

---

## 3. Paso a paso de cómo se hizo

### Paso 1 — La aplicación (`api/app.py`)

Una API Flask con cuatro rutas:

| Método y ruta | Para qué sirve |
|---|---|
| `GET /` | Página de bienvenida; confirma que la API responde |
| `GET /salud` | Le pregunta su versión a PostgreSQL — **prueba la comunicación** |
| `POST /mensajes` | Guarda un mensaje en la base de datos |
| `GET /mensajes` | Devuelve todos los mensajes guardados |

Dos decisiones importantes dentro del código:

- **`app.run(host="0.0.0.0", port=5000)`.** Por defecto Flask escucha solo en `127.0.0.1`, lo que dentro de un contenedor lo dejaría incomunicado. Con `0.0.0.0` acepta conexiones que vienen de fuera del contenedor.
- **Una función que espera a la base de datos.** PostgreSQL tarda unos segundos en aceptar conexiones. `esperar_base_de_datos()` reintenta hasta 30 veces antes de rendirse, para que la API no se muera en el primer intento.

### Paso 2 — La receta de la imagen (`api/Dockerfile`)

```
FROM python:3.12-slim      # imagen base ligera
WORKDIR /app               # carpeta de trabajo dentro del contenedor
COPY requirements.txt .    # primero las dependencias...
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .              # ...y después el código
ENV PYTHONUNBUFFERED=1     # para que los print() salgan en los logs
```

```
EXPOSE 5000
CMD ["python", "app.py"]   # qué se ejecuta al arrancar
```

**¿Por qué se copia `requirements.txt` antes que `app.py`?** Por la **caché de capas**. Docker guarda el resultado de cada instrucción. Si solo cambio `app.py`, Docker reutiliza la capa donde ya instaló las librerías en vez de descargarlas otra vez. En la práctica esto se notó muchísimo: la primera construcción tardó **19,8 segundos** y la segunda **0,5 segundos**, con todos los pasos marcados como `CACHED`.

### Paso 3 — La orquestación (`docker-compose.yml`)

Aquí se define todo lo demás: los dos servicios, la red que comparten, el volumen donde persisten los datos y el orden de arranque.

```
services:
  db:
    image: postgres:16-alpine
    container_name: semana3-db
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - datos-db:/var/lib/postgresql/data
    networks: [red-app]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    build: ./api
    container_name: semana3-api
    environment:
      DB_HOST: db          # <-- NOMBRE DEL SERVICIO, no localhost
      DB_PORT: 5432
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
    ports:
      - "${API_PORT}:5000"
    depends_on:
      db:
        condition: service_healthy
    networks: [red-app]
```

```
networks:
  red-app:
    driver: bridge

volumes:
  datos-db:
```

Puntos que vale la pena señalar:

- `healthcheck` + `depends_on: condition: service_healthy`**.** Docker pregunta cada 5 segundos si PostgreSQL ya acepta conexiones, y no arranca la API hasta que la respuesta es afirmativa. Sin esto, la API arrancaría antes de tiempo y fallaría.
- **El volumen** `datos-db`**.** Guarda la información fuera del contenedor, así que los mensajes sobreviven aunque se borre y se vuelva a crear.
- `db` **no tiene** `ports:`, intencionalmente, como se explicó arriba.

### Paso 4 — Las credenciales (`.env`)

Las contraseñas **no están escritas en el código ni en el compose**. Viven en un archivo `.env` que Docker Compose lee automáticamente:

```
POSTGRES_USER=usuario_app
POSTGRES_PASSWORD=...
POSTGRES_DB=semana3
API_PORT=5000
```

| Archivo | ¿Se sube a GitHub? | Para qué |
|---|---|---|
| `.env` | ❌ **No** | Credenciales reales de mi máquina |
| `.env.example` | ✅ Sí | Plantilla: dice qué variables hacen falta, sin revelar los valores |

El `.gitignore` incluye la línea `.env`, de modo que Git lo ignora. Se verificó con `git check-ignore .env` y buscando la contraseña dentro de los archivos que sí se subían: no aparece en ninguno.

---

## 4. Cómo levantar el proyecto

```
git clone https://github.com/SebasVelasquez/docker-semana3.git
cd docker-semana3

cp .env.example .env          # crear el archivo de credenciales
# (editar .env y poner una contraseña propia)

docker compose up --build -d  # construir y levantar los dos contenedores
```

Para comprobar y apagar:

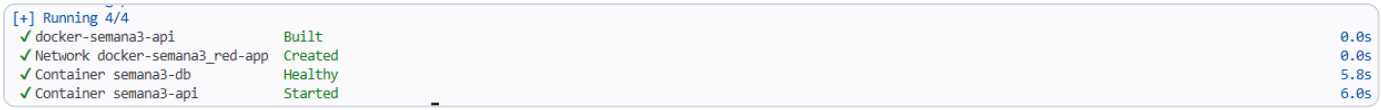
```
docker ps                     # ver los dos contenedores corriendo
./probar.sh                   # ejecutar todas las pruebas
docker compose down           # apagar
docker compose down -v        # apagar y borrar también los datos
```

---

## 5. Evidencias

### Evidencia 1 — Los dos contenedores se construyen y arrancan

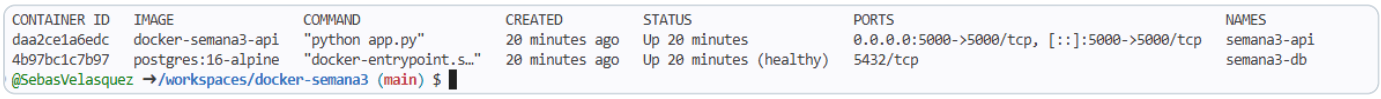
Resultado de `docker compose up --build -d`:




La imagen de la API se construye, la red `red-app` se crea, y los dos contenedores arrancan. Nótese que `semana3-db` aparece como **`Healthy`**: el `healthcheck` pasó, y solo entonces arrancó `semana3-api`.

### Evidencia 2 — Los dos contenedores están corriendo

Resultado de `docker ps`:




Lo que hay que mirar en esta captura:

| Contenedor | Imagen | Estado | Puertos |
|---|---|---|---|
| `semana3-api` | `docker-semana3-api` | `Up` | `0.0.0.0:5000->5000/tcp` |
| `semana3-db` | `postgres:16-alpine` | `Up (healthy)` | `5432/tcp` |

La diferencia en la columna **PORTS** es la clave: la API tiene `0.0.0.0:5000->5000/tcp`, es decir, está publicada hacia mi computador. La base de datos solo dice `5432/tcp`, **sin** redirección: no es alcanzable desde fuera, únicamente desde dentro de la red de Docker.

### Evidencia 3 — La API se comunica con la base de datos

Pruebas con `curl` sobre `GET /salud` y `POST /mensajes`:

```
{
  "api": "ok",
  "base_de_datos": "ok",
  "host_base_de_datos": "db",
  "version_postgres": "PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit"
}
{
  "guardado": true,
  "mensaje": {
    "creado_en": "Tue, 10 Nov 2026 12:29:17 GMT",
    "id": 3,
    "texto": "Evidencia para la entrega"
  }
}
@SebasVelasquez ➜/workspaces/docker-semana3 (main) $
```


La primera respuesta es la más importante:

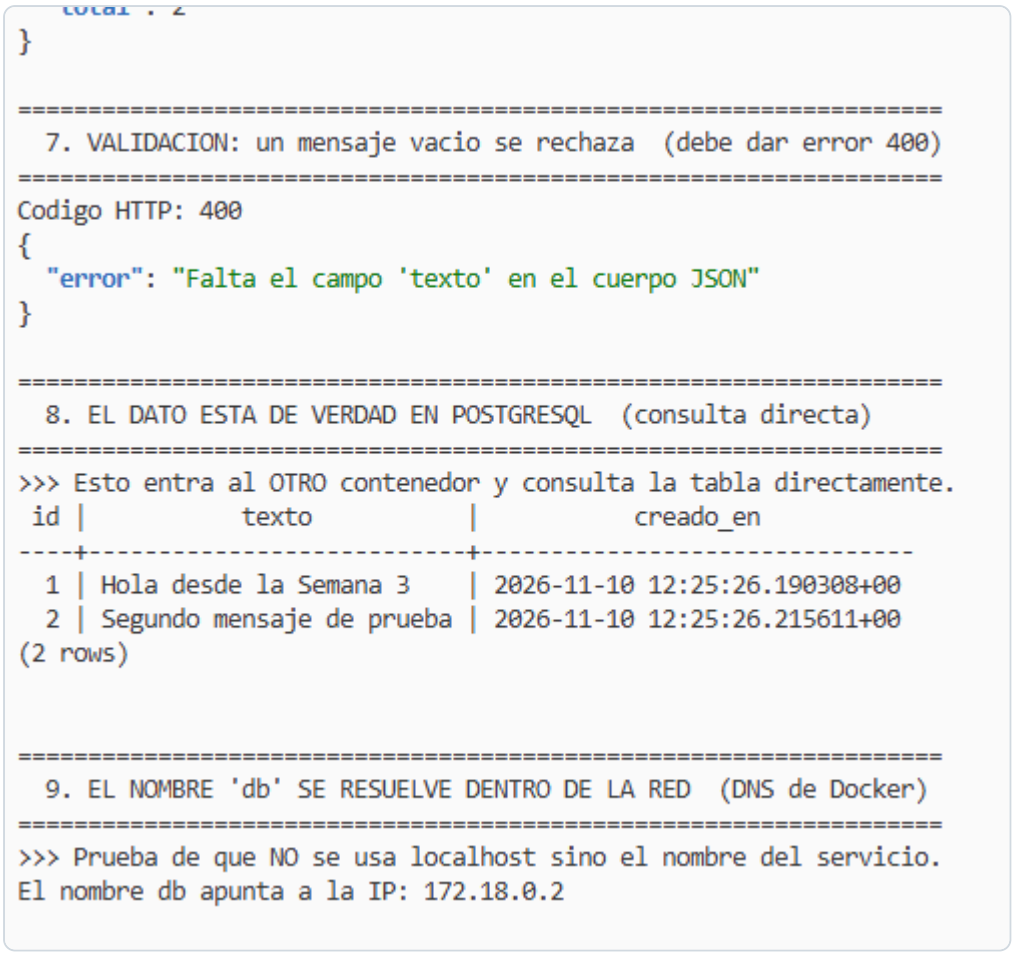

```
{
  "api": "ok",
  "base_de_datos": "ok",
  "host_base_de_datos": "db",
  "version_postgres": "PostgreSQL 16.15 on x86_64-pc-linux-musl, ..."
}
```


La API se conectó al host `db` —el nombre del servicio— y PostgreSQL le respondió con su número de versión. Esa versión no puede inventarse: viene del otro contenedor.

La segunda respuesta confirma que el mensaje se guardó, con el `id` que generó PostgreSQL y la marca de tiempo:


```
{ "guardado": true,
  "mensaje": { "id": 3, "texto": "Evidencia para la entrega",
               "creado_en": "Tue, 10 Nov 2026 12:29:17 GMT" } }
```

### Evidencia 4 — El dato está realmente en PostgreSQL, y el DNS funciona



```
  total : 2
}

================================================================
  7. VALIDACION: un mensaje vacio se rechaza  (debe dar error 400)
================================================================
Codigo HTTP: 400
{
  "error": "Falta el campo 'texto' en el cuerpo JSON"
}

================================================================
  8. EL DATO ESTA DE VERDAD EN POSTGRESQL  (consulta directa)
================================================================
>>> Esto entra al OTRO contenedor y consulta la tabla directamente.
 id |           texto           |           creado_en
----+---------------------------+-------------------------------
  1 | Hola desde la Semana 3    | 2026-11-10 12:25:26.190308+00
  2 | Segundo mensaje de prueba | 2026-11-10 12:25:26.215611+00
(2 rows)


================================================================
  9. EL NOMBRE 'db' SE RESUELVE DENTRO DE LA RED  (DNS de Docker)
================================================================
>>> Prueba de que NO se usa localhost sino el nombre del servicio.
El nombre db apunta a la IP: 172.18.0.2
```

Esta captura cierra la demostración con dos comprobaciones:

**Prueba 8** — se entra al contenedor de la base de datos y se consulta la tabla directamente, sin pasar por la API:

```
docker compose exec db psql -U usuario_app -d semana3 \
  -c "SELECT id, texto, creado_en FROM mensajes ORDER BY id;"
```

Aparecen los dos mensajes enviados por `curl`. Es decir, los datos que entraron por el contenedor de la API **terminaron físicamente dentro del contenedor de PostgreSQL**. Si no hubiera comunicación entre ambos, esta tabla estaría vacía.

**Prueba 9** — desde dentro del contenedor de la API se resuelve el nombre `db`:

```
docker compose exec api python -c "import socket; print(socket.gethostbyname('db'))"
```

Devuelve `172.18.0.2`, una dirección IP privada de la red `red-app`. Ese nombre no existe en internet ni en mi computador: lo crea el DNS interno de Docker a partir del nombre del servicio en `docker-compose.yml`.

De regalo, en la misma captura se ve la **prueba 7**: enviar un mensaje vacío devuelve `400` con un mensaje de error, lo que demuestra que la API valida lo que recibe.

---

## 6. Un error que apareció al ejecutar, y cómo se corrigió

El proyecto se ejecutó en **GitHub Codespaces**. La primera corrida falló parcialmente:

```
./probar.sh: line 42: python3: command not found
```

**Causa.** El script `probar.sh` usaba `python3 -m json.tool` para mostrar el JSON con sangría, pero la imagen base del entorno (`ubuntu-24.04`) **no trae Python instalado**. Las peticiones de `curl` sí funcionaban —los datos llegaban a la base de datos—, pero las respuestas no se alcanzaban a imprimir.

**Corrección.** Se cambió el script para que use la herramienta que esté disponible en cada máquina, en vez de dar por hecho que hay Python:

```
if command -v jq >/dev/null 2>&1; then
  formatear_json() { jq .; }
elif command -v python3 >/dev/null 2>&1; then
  formatear_json() { python3 -m json.tool; }
else
  formatear_json() { cat; echo; }   # feo, pero nunca se pierde la respuesta
fi
```

**Lección.** El código estaba correctamente escrito y la sintaxis era válida; el fallo solo apareció al ejecutarlo de verdad contra un Docker real. No se puede dar por hecho qué herramientas trae una imagen.

---

## 7. Estructura del repositorio

```
docker-semana3/
├── .devcontainer/
│   ├── devcontainer.json       # prepara Docker en GitHub Codespaces
│   └── preparar.sh             # genera el .env al crear el entorno
├── api/
│   ├── Dockerfile              # receta de la imagen de la API
│   ├── app.py                  # la aplicación Flask
│   └── requirements.txt        # Flask + psycopg2-binary
├── evidencias/                 # las capturas de este documento
├── .env                        # credenciales reales (NO se sube)
├── .env.example                # plantilla de credenciales (sí se sube)
├── .gitattributes              # fuerza finales de línea LF
├── .gitignore                  # ignora .env
├── docker-compose.yml          # los 2 servicios, la red y el volumen
├── probar.sh                   # ejecuta las 9 pruebas
├── ENTREGA.md                  # este documento
└── README.md                   # documentación del proyecto
```

---

## 8. Resumen de lo que se cumplió

| Requisito de la actividad | Estado |
|---|---|
| Dos contenedores construidos con Docker | ✅ `semana3-api` y `semana3-db` |
| Los contenedores se comunican entre sí | ✅ Demostrado en las evidencias 3 y 4 |
| Dockerfile de la API | ✅ `api/Dockerfile` |
| `docker-compose.yml` que levanta ambos | ✅ Con red, volumen y healthcheck |
| Misma red, comunicación por nombre de servicio | ✅ Red `red-app`, host `db` |
| Credenciales en `.env`, con `.env.example` | ✅ Y `.env` excluido en `.gitignore` |
| Endpoint que guarda y consulta en la base de datos | ✅ `POST` y `GET /mensajes` |
| Ejecución con `docker compose up --build` | ✅ Evidencia 1 |
| Pruebas con `curl` y corrección de errores | ✅ Evidencias 3 y 4; error corregido en la sección 6 |
| Evidencia de `docker ps` | ✅ Evidencia 2 |
| README con la documentación | ✅ `README.md` |
| Cambios subidos a GitHub | ✅ https://github.com/SebasVelasquez/docker-semana3 |